

# Agent Engine (ADK) PSC SWP 整合

來源：<https://codelabs.developers.google.com/agent-engine-psc-interface-swp?hl=zh-tw>

步驟數：15

在本教學課程中，您將瞭解如何設定及驗證 Private Service Connect 介面，並透過 Agent Engine 指定 Secure Web Proxy 做為網際網路輸出路徑。

---

# 目錄

1. [1. 簡介](#)
2. [2. 事前準備](#)
3. [3. 消費者設定](#)
4. [4. 建立 Secure Web Proxy](#)
5. [5. Private Service Connect 網路連結](#)
6. [6. 私人 DNS 區域](#)
7. [7. 為虛擬私有雲網路建立防火牆政策，確保威脅情資：](#)
8. [8. 建立 Jupyter Notebook](#)
9. [9. 建立 Vertex AI Workbench 執行個體](#)
10. [10. 更新 Vertex AI 服務代理](#)
11. [11. 部署 Agent Engine](#)
12. [12. PSC 介面驗證](#)
13. [13. SWP - Cloud Logging 驗證](#)
14. [14. 清理](#)
15. [15. 恭喜](#)

步驟 1 · 約 0 分鐘

Agent Engine (ADK) PSC SWP 整合

## 程式碼研究室簡介

subject上次更新時間：3月 16, 2026

account\_circle作者：Harika Rama Tulasi Karatapu ,Deepak Michael, Harkanwal Bedi

### 1. 簡介

Private Service Connect 介面是一種資源，可讓供應商虛擬私有雲 (VPC) 網路啟動與用戶虛擬私有雲網路中各種目的地的連線。供應商和用戶網路可以位於不同專案和機構中。

如果網路連結接受來自 Private Service Connect 介面的連線，Google Cloud 會從網路連結指定的消費者子網路，為介面分配 IP 位址。消費者和生產者網路已連線，可使用內部 IP 位址通訊。

網路連結與 Private Service Connect 介面之間的連線，類似於 Private Service Connect 端點與服務連結之間的連線，但有兩項主要差異：

- 網路連結可讓供應商網路啟動與消費者網路的連線 (代管服務輸出)，而端點則可讓消費者網路啟動與供應商網路的連線 (代管服務輸入)。
- Private Service Connect 介面連線是可遞移的。也就是說，生產端網路可以與連線至消費端網路的其他網路通訊。

## Vertex AI PSC 介面可連線性考量

- PSC 介面可將流量轉送至 VPC 網路所瞭解的 VPC 或內部部署目的地。
- 如要限制 Agent Engine 所用網路附件與虛擬私有雲網路的連線範圍，最佳做法是實作輸出防火牆規則。
- 如要限制源自 Agent Engine 網路附件子網路的網路輸出流量範圍，請部署虛擬私有雲輸出防火牆規則。這項規則會明確允許從 Agent Engine 到 SWP 的流量，並拒絕所有其他輸出流量。

## Vertex AI PSC 介面 VPC-SC 注意事項

- 即使已啟用 VPC Service Controls，您仍須在客戶虛擬私有雲中提供網際網路輸出連線，Agent Engine PSC 介面才能運作。

## Secure Web Proxy

[Secure Web Proxy](#) 是一項受管理的原生雲端服務，可讓您精細控管及保護外送流量 (HTTP/HTTPS)。這項服務可做為中央閘道，讓您對從透過 PSC 介面部署的 Agent Engine 啟動的連線，強制執行安全政策，連線對象包括 VM、GKE、網際網路和多雲端環境等 VPC 資源。

### 解決的問題

- **防止資料竊取：**封鎖未經授權的上傳作業或與惡意網站的通訊。
- **強制執行規範：**確保輸出流量符合貴機構的安全性和資料處理政策。
- **降低營運負擔：**Secure Web Proxy 是全代管服務，因此您不必部署、擴充或維護自己的 Proxy VM。
- **提供深入的能見度：**可檢查傳輸層安全標準 (TLS) 加密的流量，偵測隱藏的威脅。

如需更多資訊，請參閱下列資源：

[部署代理程式 | Vertex AI 生成式 AI | Google Cloud](#)

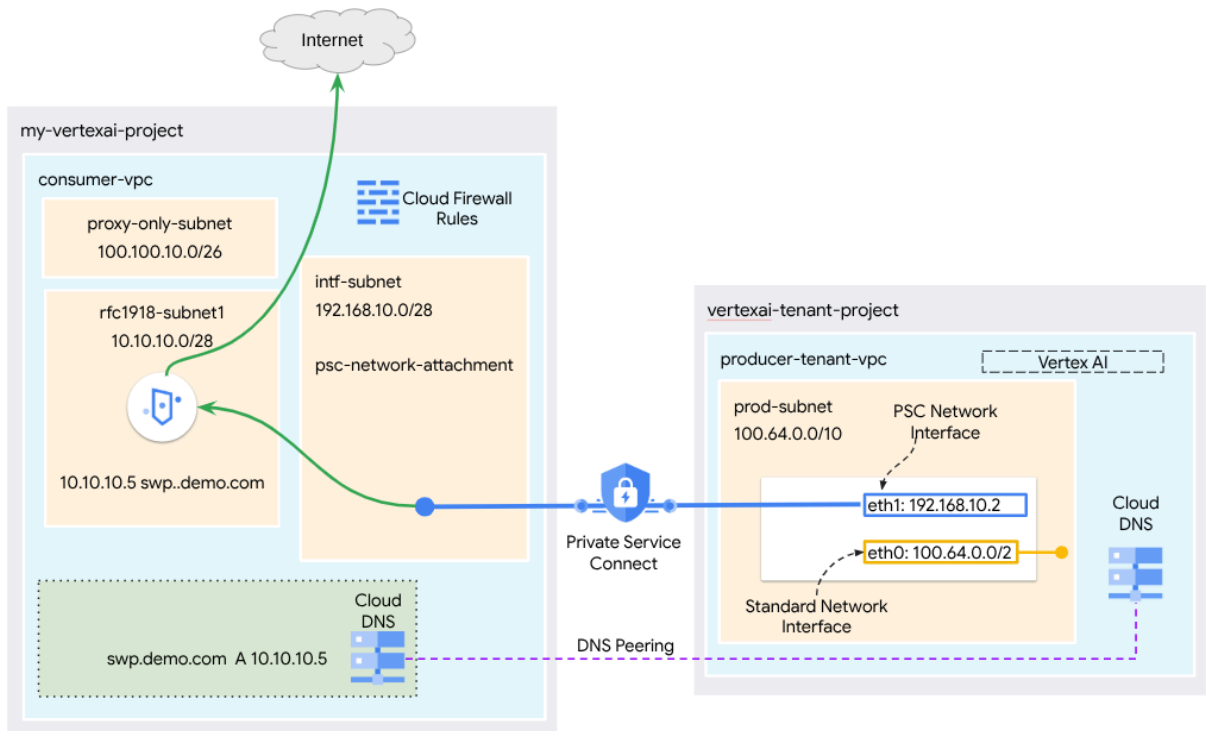
[為 Vertex AI 資源設定 Private Service Connect 介面 | Google Cloud](#)

# 建構項目

在本教學課程中，您將使用 ADK 程式庫，建構透過 Private Service Connect (PSC) 介面部署的完整 Agent Engine，並整合 SWP，以執行下列操作：

- 在 Agent Engine 中部署 DNS 對等互連，以解析 Proxy 設定中使用的 SWP 完整網域名稱。
- 透過部署在消費者 VPC 中的安全網頁 Proxy，允許連線至公開網站 (<https://api.frankfurter.app/>)，並使用 RFC1918 位址。
- 允許從網路附件子網路傳輸至 SWP 的流量，並拒絕所有其他流量。

圖 1



注意：本教學課程會根據圖示拓撲提供設定和驗證步驟，請視貴機構的需求修改程序。

## 課程內容

- 如何建立網路連結
- 生產者如何使用網路連結建立 PSC 介面
- 如何使用 DNS 對接，建立從生產端到消費端的通訊
- 如何部署及使用 SWP 進行網際網路輸出
- 如何定義輸出防火牆規則，減少 Agent Engine 網路可連線範圍

## 軟硬體需求

Google Cloud 專案

IAM 權限

- [Compute 網路管理員](#) (roles/compute.networkAdmin)
- [Compute 執行個體管理員](#) (roles/compute.instanceAdmin)

- [Compute 安全管理員](#) (roles/compute.securityAdmin)
  - [DNS 管理員](#) (roles/dns.admin)
  - [受 IAP 保護的通道使用者](#) (roles/iap.tunnelResourceAccessor)
  - [記錄管理員](#) (roles/logging.admin)
  - [Notebooks 管理員](#) (roles/notebooks.admin)
  - [專案 IAM 管理員](#) (roles/resourcemanager.projectIamAdmin)
  - [服務帳戶管理員](#) (roles/iam.serviceAccountAdmin)
  - [服務使用情形管理員](#) (roles/serviceusage.serviceUsageAdmin)
-

步驟 2 · 約 2 分鐘

## 2. 事前準備

### 更新專案以支援教學課程

本教學課程會使用 `$variables`，協助您在 Cloud Shell 中實作 `gcloud` 設定。

在 Cloud Shell 中執行下列操作：

```
gcloud config set project [YOUR-PROJECT-NAME]
projectid=YOUR-PROJECT-NAME
echo $projectid
```

### 啟用 API

在 Cloud Shell 中執行下列操作：

```
gcloud services enable "compute.googleapis.com"
gcloud services enable "aiplatform.googleapis.com"
gcloud services enable "dns.googleapis.com"
gcloud services enable "notebooks.googleapis.com"
gcloud services enable "storage.googleapis.com"
gcloud services enable "iap.googleapis.com"
gcloud services enable "networksecurity.googleapis.com"
gcloud services enable "networkservices.googleapis.com"
gcloud services enable "cloudresourcemanager.googleapis.com"
```

確認 API 已成功啟用

```
gcloud services list --enabled
```

---

## 3. 消費者設定

### 建立 Consumer VPC

這個虛擬私有雲位於客戶專案中。這個虛擬私有雲中會建立下列資源

- 消費者子網路
- 網路連結子網路
- 僅限 Proxy 的子網路
- 防火牆規則
- Cloud DNS

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks create consumer-vpc --project=$projectid --subnet-mode=custom
```

### 建立消費者子網路

在 Cloud Shell 中，為 SWP 建立子網路：

```
gcloud compute networks subnets create swp-subnet --project=$projectid --range=10.10.10.0/28  
--network=consumer-vpc --region=us-central1 --enable-private-ip-google-access
```

### 建立 Private Service Connect 網路連結子網路

在 Cloud Shell 中，為 PSC 網路附件建立子網路：

```
gcloud compute networks subnets create intf-subnet --project=$projectid --  
range=192.168.10.0/28 --network=consumer-vpc --region=us-central1 --enable-private-ip-google-  
access
```

### 建立區域 Proxy 子網路

在 Cloud Shell 中，建立 Envoy 型產品 (例如 Secure Web Proxy 和區域性內部/外部應用程式負載平衡器) 所需的 Proxy 專用子網路。–purpose 旗標必須設為 REGIONAL\_MANAGED\_PROXY：

```
gcloud compute networks subnets create proxy-subnet \  
--purpose=REGIONAL_MANAGED_PROXY \  
--role=ACTIVE \  
--region=us-central1 \  
--network=consumer-vpc \  
--range=100.100.10.0/26
```

### 建立筆記本子網路

在 Cloud Shell 中，為筆記本執行個體建立子網路：

```
gcloud compute networks subnets create notebook-subnet --project=$projectid --  
range=192.168.20.0/28 --network=consumer-vpc --region=us-central1 --enable-private-ip-google-  
access
```

---



步驟 4 · 約 0 分鐘

## 4. 建立 Secure Web Proxy

安全網路 Proxy 的明確模式 (或明確 Proxy 轉送模式) 是一種部署方法，必須明確設定用戶端工作負載，才能將 SWP 的內部 IP 位址或完整網域名稱和連接埠做為轉送 Proxy。

這項政策會根據工作階段比對 `host() == 'api.frankfurter.app'` 和應用程式比對 `request.method == 'GET'`，制定規則來控管透過 Secure Web Proxy 的流量。

在下列步驟中，請務必將 `YOUR-PROJECT-ID` 修改為您的專案 ID

在 Cloud Shell 中建立 `policy.yaml` 檔案：

```
cat > policy.yaml << EOF
name: projects/$projectid/locations/us-central1/gatewaySecurityPolicies/my-swp-policy
description: "My basic SWP policy"
EOF
```

在 Cloud Shell 中匯入政策：

```
gcloud network-security gateway-security-policies import my-swp-policy \
  --source=policy.yaml \
  --location=us-central1
```

### 建立 Secure Web Proxy 規則

在政策中定義規則，指定允許或拒絕的流量。系統會依優先順序評估規則。

在 Cloud Shell 中，建立 `rule.yaml` 檔案，允許存取代理程式引擎使用的網際網路端點 `api.frankfurter.app`：

```
cat > rule.yaml << EOF
name: "projects/$projectid/locations/us-central1/gatewaySecurityPolicies/my-swp-policy/rules/allow-example"
description: "Allow frankfurter API"
enabled: true
priority: 10
basicProfile: ALLOW
sessionMatcher: "host() == 'api.frankfurter.app'"
EOF
```

**注意：**如未啟用 TLS 檢查功能，應用程式比對器 `request.method == 'GET'` 只會比對 HTTP 流量，不會比對 HTTPS 流量

在 Cloud Shell 中產生安全性政策規則：

```
gcloud network-security gateway-security-policies rules import allow-example \
  --source=rule.yaml \
  --location=us-central1 \
  --gateway-security-policy=my-swp-policy
```

## 建立 Secure Web Proxy 規則

您必須建立以明確路由模式部署的 SWP 執行個體，以便 Agent Engine 在 ADK Proxy 設定中指定 SWP 的 IP 位址或 FQDN，如閘道 YAML 檔案中所定義。這項設定也會將執行個體連結至對應的政策、網路和子網路。

在 Cloud Shell 中，建立用於部署 SWP 的 gateway.yaml 檔案。

請務必更新下列變數，填入環境詳細資料，然後儲存 YAML 檔案：PROJECT\_ID、REGION、NETWORK\_NAME 和 PROXY\_ONLY\_SUBNET\_NAME。指定的 8888 埠是外部通道埠，會對應至 Agent Engine 內的 Proxy 設定。

```
cat > gateway.yaml << EOF
name: "projects/$projectid/locations/us-central1/gateways/my-swp-instance"
type: SECURE_WEB_GATEWAY
ports: [8888]
addresses: ["10.10.10.5"]
gatewaySecurityPolicy: "projects/$projectid/locations/us-central1/gatewaySecurityPolicies/my-swp-policy"
network: "projects/$projectid/global/networks/consumer-vpc"
subnetwork: "projects/$projectid/regions/us-central1/subnetworks/swp-subnet"
routingMode: EXPLICIT_ROUTING_MODE
EOF
```

在 Cloud Shell 中匯入閘道：

```
gcloud network-services gateways import my-swp-instance \
  --source=gateway.yaml \
  --location=us-central1
```

## 5. Private Service Connect 網路連結

網路連結是區域資源，代表 Private Service Connect 介面的用戶端。您會將單一子網路與網路連結建立關聯，而生產端會從該子網路將 IP 指派給 Private Service Connect 介面。子網路必須與網路連結位於同一地區。網路連結必須與生產者服務位於相同區域。

### 建立網路連結

在 Cloud Shell 中建立網路連結。

```
gcloud compute network-attachments create psc-network-attachment \
  --region=us-central1 \
  --connection-preference=ACCEPT_AUTOMATIC \
  --subnets=intf-subnet
```

### 列出網路連結

在 Cloud Shell 中列出網路連結。

```
gcloud compute network-attachments list
```

### 說明網路連結

在 Cloud Shell 中，說明網路附件。

```
gcloud compute network-attachments describe psc-network-attachment --region=us-central1
```

請記下 **PSC 網路連結名稱** `psc-network-attachment`，供應商建立 Private Service Connect 介面時會用到這個名稱。  
如要在 Cloud 控制台中查看 PSC 網路附件網址，請前往下列位置：

Network Services → Private Service Connect → Network Attachment → `psc-network-attachment`

Network Services ☆

Load balancing

Cloud DNS

Cloud CDN

Cloud NAT

Cloud Service Mesh (Traff...

Service Directory

Cloud Domains

**Private Service Connect**

SSL policies

Service Extensions

← Network attachment details Edit Delete

psc-network-attachment

|                       |                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------|
| Network attachment    | projects/dec30-run1-agent/regions/us-central1/networkAttachments/psc-network-attachment |
| Region                | us-central1                                                                             |
| Network               | <a href="#">consumer-vpc</a>                                                            |
| Subnetworks           | <a href="#">intf-subnet</a>                                                             |
| Connection preference | Accept automatically                                                                    |
| Tags                  | —                                                                                       |

Connected projects

Filter Filter projects ?

| <input type="checkbox"/> | Name ↑ | Status | Connections | Actions |
|--------------------------|--------|--------|-------------|---------|
| No rows to display       |        |        |             |         |

步驟 6 · 約 3 分鐘

## 6. 私人 DNS 區域

您將為 `demo.com` 建立 Cloud DNS 區域，並填入指向 SWP IP 位址的 A 記錄。稍後，系統會在 Agent Engine 中部署 DNS 對等互連，允許存取消費者的 DNS 記錄。

在 Cloud Shell 中執行下列指令，建立 DNS 名稱 `demo.com`。

```
gcloud dns --project=$projectid managed-zones create private-dns-codelab --description="" --  
dns-name="demo.com." --visibility="private" --  
networks="https://compute.googleapis.com/compute/v1/projects/$projectid/global/networks/consumer-vpc"
```

取得並儲存用於 DNS A 記錄的 SWP IP 位址。

在 Cloud Shell 中，對 `swp` 執行描述，`my-swp-instance`：

```
gcloud network-services gateways describe my-swp-instance --location=us-central1
```

在 Cloud Shell 中，為 SWP (`swp.demo.com`) 建立記錄集，並根據環境的輸出內容更新 IP 位址。

```
gcloud dns --project=$projectid record-sets create swp.demo.com. --zone="private-dns-codelab"  
--type="A" --ttl="300" --rrdatas="10.10.10.5"
```

防火牆設定

## 建立 Cloud Firewall 規則，允許從 PSC 介面存取

在下一節中，建立防火牆規則，允許來自 PSC 網路附件的流量存取 Consumer VPC 中的 SWP 子網路。為提高安全性，您可以將 SWP IP 位址指定為唯一目的地。

在 Cloud Shell 中，建立允許從網路附件存取 SWP 的輸出防火牆規則：

```
gcloud compute firewall-rules create allow-access-to-swp \  
  --network=consumer-vpc \  
  --action=ALLOW \  
  --rules=ALL \  
  --direction=EGRESS \  
  --priority=1000 \  
  --source-ranges="192.168.10.0/28" \  
  --destination-ranges="10.10.10.5/32" \  
  --enable-logging
```

在 Cloud Shell 中，建立拒絕所有來自網路附件流量的輸出防火牆規則：

```
gcloud compute firewall-rules create deny-all \  
  --network=consumer-vpc \  
  --action=DENY \  
  --rules=ALL \  
  --direction=EGRESS \  
  --priority=65534 \  
  --source-ranges="192.168.10.0/28" \  
  --destination-ranges="0.0.0.0/0" \  
  --enable-logging
```

---

## 7. 為虛擬私有雲網路建立防火牆政策，確保威脅情資：

在下一節中，請建立防火牆政策，以便利用 Google 管理的威脅清單，在 SWP 接收流量前封鎖已知的惡意網站。

在 Cloud Shell 中建立全域防火牆政策：

```
gcloud compute network-firewall-policies create psc-secure-policy \
  --global \
  --description="Policy to protect VPC with Threat Intelligence"
```

在 Cloud Shell 中，將政策與虛擬私有雲建立關聯：

```
gcloud compute network-firewall-policies associations create \
  --firewall-policy=psc-secure-policy \
  --network=consumer-vpc \
  --name=psc-swp-association \
  --global-firewall-policy
```

### 在 Cloud Shell 中新增威脅情報規則：

這些規則會在從代理程式啟動流量前，將流量傳送給已知的惡意行為者。在這個範例中，我們新增了「封鎖 Tor 結束節點 (輸出)」和「封鎖已知惡意 IP (輸出)」、「封鎖已知匿名 Proxy(輸出)」、「封鎖加密貨幣挖礦程式，防止未經授權使用資源 (輸出)」規則，

```
gcloud compute network-firewall-policies rules create 100 \
  --firewall-policy=psc-secure-policy \
  --action=deny \
  --direction=EGRESS \
  --dest-threat-intelligence=iplist-tor-exit-nodes \
  --layer4-configs=all \
  --enable-logging \
  --description="Block anonymous Tor traffic" \
  --global-firewall-policy
```

```
gcloud compute network-firewall-policies rules create 110 \
  --firewall-policy=psc-secure-policy \
  --action=deny \
  --direction=EGRESS \
  --dest-threat-intelligence=iplist-known-malicious-ips \
  --layer4-configs=all \
  --enable-logging \
  --description="Block known botnets and malware sources" \
  --global-firewall-policy
```

```
gcloud compute network-firewall-policies rules create 120 \  
  --firewall-policy=psc-secure-policy \  
  --action=deny \  
  --direction=EGRESS \  
  --dest-threat-intelligence=iplist-anon-proxies \  
  --layer4-configs=all \  
  --enable-logging \  
  --description="Block Known Anonymous Proxies" \  
  --global-firewall-policy
```

```
gcloud compute network-firewall-policies rules create 130 \  
  --firewall-policy=psc-secure-policy \  
  --action=deny \  
  --direction=EGRESS \  
  --dest-threat-intelligence=iplist-crypto-miners \  
  --layer4-configs=all \  
  --enable-logging \  
  --description="Block Crypto Miners (Prevent unauthorized resource usage)" \  
  --global-firewall-policy
```

---

## 8. 建立 Jupyter Notebook

下一節將引導您建立 Jupyter Notebook。這個筆記本將用於部署 Agent Engine，以明確的 Proxy 做為網際網路輸出目標。

### 建立使用者管理的服務帳戶

在下一節中，您將建立與本教學課程所用 Vertex AI Workbench 執行個體相關聯的服務帳戶。

在本教學課程中，服務帳戶會套用下列角色：

- [儲存空間管理員](#)
- [Vertex AI 使用者](#)
- [Artifact Registry 管理員](#)
- [IAM 服務帳戶使用者](#)

在 Cloud Shell 中建立服務帳戶。

```
gcloud iam service-accounts create notebook-sa \  
  --display-name="notebook-sa"
```

在 Cloud Shell 中，將服務帳戶更新為 Storage 管理員角色。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/storage.admin"
```

在 Cloud Shell 中，使用 Vertex AI 使用者角色更新服務帳戶。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/aiplatform.user"
```

在 Cloud Shell 中，更新服務帳戶，並指派 Artifact Registry 管理員角色。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/artifactregistry.admin"
```

在 Cloud Shell 中，允許筆記本服務帳戶使用 Compute Engine 預設服務帳戶。

```
gcloud iam service-accounts add-iam-policy-binding \  
  $(gcloud projects describe $(gcloud config get-value project) --  
format='value(projectNumber)')-compute@developer.gserviceaccount.com \  
  --member="serviceAccount:notebook-sa@$projectid.iam.gserviceaccount.com" \  
  --role="roles/iam.serviceAccountUser"
```

---



## 9. 建立 Vertex AI Workbench 執行個體

在下一節中，建立納入先前建立的服務帳戶 notebook-sa 的 Vertex AI Workbench 執行個體。

在 Cloud Shell 中建立 private-client 執行個體。

```
gcloud workbench instances create workbench-tutorial --vm-image-project=cloud-notebooks-managed --vm-image-family=workbench-instances --machine-type=n1-standard-4 --location=us-central1-a --subnet-region=us-central1 --subnet=notebook-subnet --disable-public-ip --shielded-secure-boot=true --shielded-integrity-monitoring=true --shielded-vtpm=true --service-account-email=notebook-sa@$projectid.iam.gserviceaccount.com
```

在現有的 Secure Web Proxy 中新增另一項規則，轉送來自這個 Notebook 執行個體的流量：

在 Cloud Shell 中，使用文字編輯器建立 rule-notebook.yaml 檔案，並務必使用專案 ID 更新 YAML

```
cat > rule-notebook.yaml << EOF
name: projects/$projectid/locations/us-central1/gatewaySecurityPolicies/my-swp-policy/rules/allow-notebook-subnet
description: Allow Internet access for notebook subnet
enabled: true
priority: 2
basicProfile: ALLOW
sessionMatcher: inIpRange(source.ip, '192.168.20.2')
EOF
```

在 Cloud Shell 中產生安全性政策規則：

```
gcloud network-security gateway-security-policies rules import allow-notebook-subnet \
  --source=rule-notebook.yaml \
  --location=us-central1 \
  --gateway-security-policy=my-swp-policy
```

---

## 10. 更新 Vertex AI 服務代理

Vertex AI 會代表您執行作業，例如從用於建立 PSC 介面的 PSC 網路連結子網路取得 IP 位址。為此，Vertex AI 會使用[服務代理程式](#) (如下所列)，該代理程式需要[網路管理員](#)權限：

`service-$projectnumber@gcp-sa-aiplatform.iam.gserviceaccount.com`

注意：更新服務代理程式權限前，請先前往 Cloud 控制台的 Vertex AI，確認已啟用 Vertex AI API。

在 Cloud Shell 中取得專案編號。

```
gcloud projects describe $projectid | grep projectNumber
```

在 Cloud Shell 中設定專案編號。

```
projectnumber=YOUR-PROJECT-NUMBER
```

在 Cloud Shell 中，為 AI Platform 建立服務帳戶。如果專案中已有服務帳戶，請略過這個步驟。

```
gcloud beta services identity create --service=aiplatform.googleapis.com --  
project=$projectnumber
```

在 Cloud Shell 中，使用 `compute.networkAdmin` 角色更新服務代理人帳戶。

```
gcloud projects add-iam-policy-binding $projectid --  
member="serviceAccount:service-$projectnumber@gcp-sa-aiplatform.iam.gserviceaccount.com" --  
role="roles/compute.networkAdmin"
```

在 Cloud Shell 中，使用 `dns.peer` 角色更新服務代理程式帳戶

```
gcloud projects add-iam-policy-binding $projectid --  
member="serviceAccount:service-$projectnumber@gcp-sa-aiplatform.iam.gserviceaccount.com" --  
role="roles/dns.peer"
```

## 更新預設服務帳戶

[授予預設服務帳戶](#) Vertex AI 的存取權。請注意，存取權變更可能需要一段時間才會生效。

在 Cloud Shell 中，使用 `aiplatform.user` 角色更新預設服務帳戶

```
gcloud projects add-iam-policy-binding $projectid \  
--member="serviceAccount:$projectnumber-compute@developer.gserviceaccount.com" \  
--role="roles/aiplatform.user"
```

## 11. 部署 Agent Engine

注意：我們會使用 GCP 主控台和 JupyterLab 筆記本，完成本節中的工作

在下一節中，您將建立執行下列工作的筆記本：

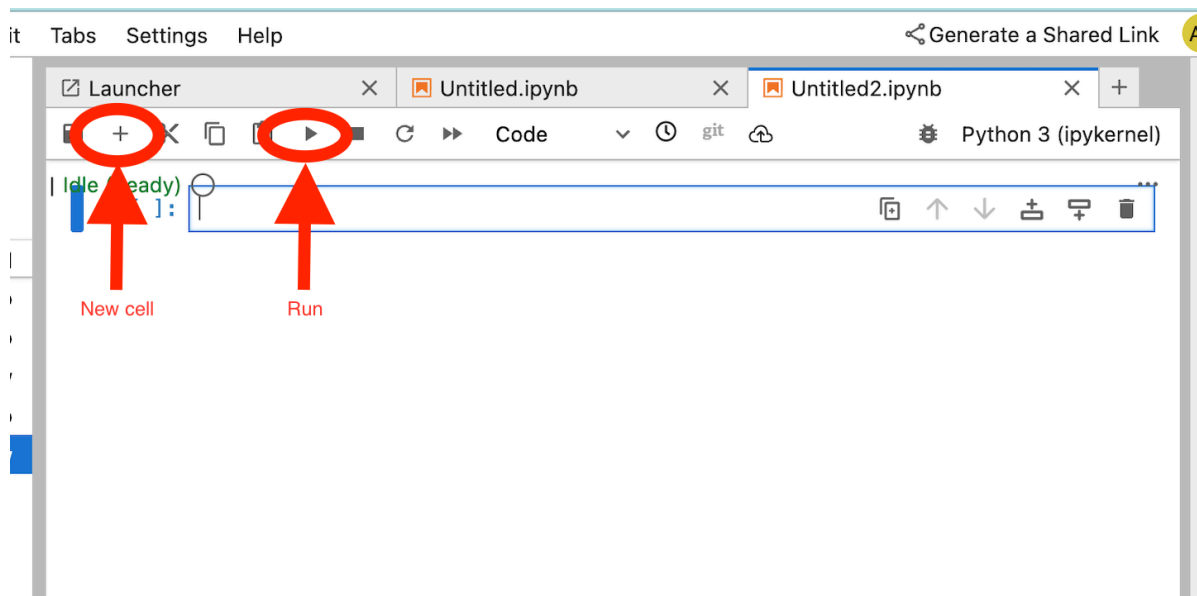
- 使用 Frankfurter API (<https://api.frankfurter.app/>) 取得匯率資料
- 參考明確的 Proxy (proxy\_server)，使用 FQDN swp.demo.com，以消費者 VPC 中的 SWP 為目標
- 定義 dnsPeeringConfigs "domain": "demo.com."

在 Vertex AI Workbench 執行個體中執行訓練工作。

- 在 Google Cloud 控制台中，前往 **Vertex AI → Workbench**
- 按一下 Vertex AI Workbench 執行個體名稱 (workbench-tutorial) 旁的「Open JupyterLab」。Vertex AI Workbench 執行個體會在 JupyterLab 中開啟。
- 選取「File > New > Notebook」
- 選取「Kernel > Python 3」

安裝必要的 Python 程式庫：安裝 Agent Engine 必要的程式庫，包括 pyyaml、google-cloud-aiplatform、cloudpickle、google-cloud-api-keys 和 langchain-google-vertexai。

在 JupyterLab 筆記本中，建立新的儲存格並執行下列程式碼，指定 SWP 的 IP 位址



```
!pip install --proxy http://10.10.10.5:8888 --upgrade google-cloud-aiplatform[agent_engines,adk]
```

根據環境，在下列程式碼片段中定義變數：

- PROJECT\_ID
- BUCKET\_NAME
- AGENT\_NAME

在本實驗室中，您將使用 BUCKET\_NAME 和 AGENT\_NAME 變數，初始化及設定全域可用的儲存空間值區

在下列章節中，系統會定義 PROXY\_SERVER (例如 swp.demo.com)，這需要 DNS 對等互連才能解析名稱。在設定中，AGENT\_PEER\_DOMAIN 會部署為 demo.com，這與 AGENT\_PEER\_NETWORK (即 consumer-vpc) 中先前步驟建立的私人 DNS 區域相符。

在 JupyterLab 筆記本中，建立新的儲存格並執行下列程式碼：

```
# --- Fundamental Project Configuration ---
PROJECT_ID = "YOUR_PROJECT_ID"
LOCATION = "us-central1" # e.g., "us-central1"
BUCKET_NAME = "YOUR_BUCKET_NAME" # A GCS bucket in the same location

# --- Agent Configuration ---
AGENT_NAME = "YOUR_AGENT_NAME"
MODEL = "gemini-2.5-flash" # Or another suitable model

# --- Network and Proxy Configuration ---
# The agent will call the Frankfurter API via this proxy
PROXY_SERVER = "http://swp.demo.com:8888"

# --- Deployment Configuration (PSC & DNS Peering) ---
# This should be a pre-existing Network Attachment
NETWORK_ATTACHMENT_NAME = f"projects/{PROJECT_ID}/regions/{LOCATION}/networkAttachments/psc-
network-attachment"
# Optional DNS Peering config
AGENT_PEER_DOMAIN = "demo.com."
AGENT_PEER_NETWORK = "consumer-vpc"

# --- Initialize Vertex AI SDK ---
import vertexai
STAGING_BUCKET = f"gs://{BUCKET_NAME}"

vertexai.init(project=PROJECT_ID, location=LOCATION, staging_bucket=STAGING_BUCKET)

print(f"Vertex AI SDK initialized for project {PROJECT_ID} in {LOCATION}.")
```

在 JupyterLab 筆記本中建立新儲存格，然後執行下列程式碼。

```
!adk create $AGENT_NAME --model=$MODEL --project=$PROJECT_ID --region=$LOCATION
```

在 JupyterLab 筆記本中建立新儲存格，然後執行下列指令，建立對應至 SWP FQDN 和通訊埠的 Proxy 變數。

```
import os
os.environ["PROXY_SERVER_URL"] = "http://swp.demo.com:8888"
```

下列程式碼儲存格示範 Agent Engine 的明確 Proxy 設定，指定 SWP 並使用對應至

os.environ["PROXY\_SERVER\_URL"] 的 PROXY\_SERVER\_TO\_USE，存取網際網路端點 api.frankfurter.app。

```
import requests
# Use the globally defined proxy server URL
proxies = {
    "http": PROXY_SERVER_TO_USE,
    "https": PROXY_SERVER_TO_USE,
}
try:
    response = requests.get(
        f"https://api.frankfurter.app/{currency_date}",
        params={"from": currency_from, "to": currency_to},
        proxies=proxies,
    )
    response.raise_for_status()
    print(response.json())
except requests.exceptions.RequestException as e: print(f"An error occurred: {e}")
```

在 JupyterLab 筆記本中建立新儲存格，然後執行下列程式碼，定義貨幣兌換目標 API (api.frankfurter.app) 的工具實作。

```

%%writefile $AGENT_NAME/agent.py
from google.adk.agents.llm_agent import Agent
import os
import requests

# Get Proxy Server URL
# This is the VM's FQDN to reach the proxy vm in the consumers network
if "PROXY_SERVER_URL" not in os.environ:
    raise ValueError("Missing required environment variable: PROXY_SERVER_URL is not set.")
PROXY_SERVER_TO_USE = os.environ["PROXY_SERVER_URL"]

# Mock tool implementation
def get_exchange_rate(
    currency_from: str = "USD",
    currency_to: str = "EUR",
    currency_date: str = "latest",
):
    """Retrieves the exchange rate between two currencies on a specified date.

    Uses the Frankfurter API (https://api.frankfurter.app/) to obtain
    exchange rate data.

    Args:
        currency_from: The base currency (3-letter currency code).
            Defaults to "USD" (US Dollar).
        currency_to: The target currency (3-letter currency code).
            Defaults to "EUR" (Euro).
        currency_date: The date for which to retrieve the exchange rate.
            Defaults to "latest" for the most recent exchange rate data.
            Can be specified in YYYY-MM-DD format for historical rates.

    Returns:
        dict: A dictionary containing the exchange rate information.
            Example: {"amount": 1.0, "base": "USD", "date": "2023-11-24",
                    "rates": {"EUR": 0.95534}}
    """
    # Use the globally defined proxy server URL
    proxies = {
        "http": PROXY_SERVER_TO_USE,
        "https": PROXY_SERVER_TO_USE,
    }

    try:
        response = requests.get(
            f"https://api.frankfurter.app/{currency_date}",
            params={"from": currency_from, "to": currency_to},
            proxies=proxies,
        )
        response.raise_for_status() # Raise an error for bad responses
        return response.json()
    except Exception as e:

```

```
        return f"An unexpected error occurred: {e}"

root_agent = Agent(
    model='gemini-2.5-flash',
    name='root_agent',
    description="Provides the currency exchange rates between two currencies",
    instruction="You are a helpful assistant that provides the currency exchange rates between two currencies. Use the 'get_exchange_rate' tool for this purpose.",
    tools=[get_exchange_rate],
)
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```
# 1. Set your variables
CURRENCY_DATE="latest"
CURRENCY_FROM="USD"
CURRENCY_TO="EUR"
PROXY_SERVER="http://swp.demo.com:8888"

# 2. Run the curl command
!curl -x "$PROXY_SERVER" "https://api.frankfurter.app/$CURRENCY_DATE?
from=$CURRENCY_FROM&to=$CURRENCY_TO"
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼，除了 DNS 對等互連外，還會叫用 Agent Engine 使用的 PSC 介面設定。

```

import json
import os

CONFIG_FILE_PATH = os.path.join(AGENT_NAME, ".agent_engine_config.json")
# Create your config as a Python dictionary ---
config_data = {
    "requirements": [
        "google-cloud-aiplatform[agent_engines,adk]",
        "requests",
    ],
    "psc_interface_config": {
        "network_attachment": NETWORK_ATTACHMENT_NAME,
        "dns_peering_configs": [
            {
                "domain": AGENT_PEER_DOMAIN,
                "target_project": PROJECT_ID,
                "target_network": AGENT_PEER_NETWORK,
            },
        ],
    },
}

# Write the dictionary to a JSON file ---
os.makedirs(AGENT_NAME, exist_ok=True) # Ensure the directory exists
with open(CONFIG_FILE_PATH, 'w') as f:
    json.dump(config_data, f, indent=4)

print(f"Successfully created {CONFIG_FILE_PATH} with your variables.")

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。



```

import json
import os

CONFIG_FILE_PATH = os.path.join(AGENT_NAME, ".agent_engine_config.json")
# Create your config as a Python dictionary ---
config_data = {

    "psc_interface_config": {
        "network_attachment": NETWORK_ATTACHMENT_NAME,
        "dns_peering_configs": [
            {
                "domain": AGENT_PEER_DOMAIN,
                "target_project": PROJECT_ID,
                "target_network": AGENT_PEER_NETWORK,
            },
        ],
    },
}

# Write the dictionary to a JSON file ---
os.makedirs(AGENT_NAME, exist_ok=True) # Ensure the directory exists
with open(CONFIG_FILE_PATH, 'w') as f:
    json.dump(config_data, f, indent=4)

print(f"Successfully created {CONFIG_FILE_PATH} with your variables.")

```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```

%%writefile $AGENT_NAME/.env

GOOGLE_CLOUD_PROJECT=PROJECT_ID
GOOGLE_CLOUD_LOCATION=us-central1
GOOGLE_GENAI_USE_VERTEXAI=1

PROXY_SERVER_URL=http://swp.demo.com:8888

```

在 JupyterLab 筆記本中，建立新的儲存格並執行下列程式碼，建立 Agent。

```

!adk deploy agent_engine $AGENT_NAME --staging_bucket=$STAGING_BUCKET --
env_file=$AGENT_NAME/.env --agent_engine_config_file=$AGENT_NAME/.agent_engine_config.json --
display_name=$AGENT_NAME

```

執行儲存格時，系統會產生推理引擎 ID。在下一個步驟中，您需要使用產生的 ID，在本範例中為

3235268984265768960。

✔ Created agent engine: projects/9315891080/locations/us-central1/reasoningEngines/3235268984265768960

在 JupyterLab 筆記本中建立新儲存格，然後執行下列指令，請務必根據先前的輸出內容更新專案編號和 Agent Engine 推理 ID：

```
from vertexai import agent_engines
remote_app = agent_engines.get("projects/PROJECT_NUMBER/locations/us-
central1/reasoningEngines/ENTER_YOUR_ID")
```

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼。

```

def print_event_nicely_with_thoughts(event):
    """
    Parses and prints streaming query events, including thoughts.
    """
    try:
        content = event.get('content', {})
        role = content.get('role')
        parts = content.get('parts', [{}])

        if not parts:
            print("...")
            return

        part = parts[0] # Get the first part

        # Event 1: Model is thinking (calling a tool or just text)
        if role == 'model':

            # Check for and print any explicit 'thought' text
            if 'thought' in part:
                print(f"🧠 Thought: {part['thought']}")

            # Check for a function call
            if 'function_call' in part:
                # If we haven't *already* printed an explicit thought,
                # print a generic one.
                if 'thought' not in part:
                    print(f"🧠 Thinking... (decided to use a tool)")

                call = part['function_call']
                print(f"🔧 Tool Call: {call.get('name')}()")
                print(f"    Args: {call.get('args')}")

            # Check for the final text answer
            elif 'text' in part:
                text = part.get('text', '')
                print(f"\n😊 Model: {text}")

        # Event 2: The tool returns its result
        elif role == 'user' and 'function_response' in part:
            resp = part['function_response']
            print(f"⚙️ Tool Response (from {resp.get('name')})")
            print(f"    Output: {resp.get('response')}")

        # Other event types (like progress messages)
        else:
            print("...") # Show progress for other events

    except Exception as e:
        print(f"Error processing event: {e}")
        # print(f"Raw event: {event}") # Uncomment to debug

```

```
for event in remote_app.stream_query(  
    user_id="u_456",  
    # session_id=remote_session["id"],  
    message="Provide USD to INR conversion rate",  
):  
    print_event_nicely_with_thoughts(event)
```

以下是成功執行的範例，根據美元兌印度盧比的匯率，驗證透過 SWP 連線至公開端點 [api.frankfurter.app](https://api.frankfurter.app)。

🧠 Thinking... (decided to use a tool)  
🔑 Tool Call: `get_exchange_rate()`  
    Args: {'currency\_from': 'USD', 'currency\_to': 'INR'}  
⚙️ Tool Response (from `get_exchange_rate`):  
    Output: {'amount': 1.0, 'base': 'USD', 'date': '2025-12-29', 'rates':  
    {'INR': 89.9}}

💬 Model: The conversion rate from USD to INR is 89.9.

---

## 12. PSC 介面驗證

您也可以前往下列位置，查看 Agent Engine 使用的網路附加 IP：

「網路服務」→「Private Service Connect」→「網路連結」→「psc-network-attachment」

選取租戶專案 (專案名稱結尾為「-tp」)

psc-network-attachment

|                       |                                                                                         |
|-----------------------|-----------------------------------------------------------------------------------------|
| Network attachment    | projects/dec30-run1-agent/regions/us-central1/networkAttachments/psc-network-attachment |
| Region                | us-central1                                                                             |
| Network               | <a href="#">consumer-vpc</a>                                                            |
| Subnetworks           | <a href="#">intf-subnet</a>                                                             |
| Connection preference | Accept automatically                                                                    |
| Tags                  | —                                                                                       |

Connected projects

Filter

Filter projects

| <input type="checkbox"/> | Name ↑                                | Status   | Connections | Actions |
|--------------------------|---------------------------------------|----------|-------------|---------|
| <input type="checkbox"/> | <a href="#">q6cfb5e73b968df52p-tp</a> | Accepted | 2           |         |

醒目顯示的欄位表示 Agent Engine 從 PSC 網路附件使用的 IP 位址。

q6cfb5e73b968df52p-tp

|             |          |
|-------------|----------|
| Status      | Accepted |
| Connections | 2        |

Endpoints

Filter

Filter endpoints

| Subnetwork ↑ | IPv4 address | IPv6 address | Alias IP ranges | Status   |
|--------------|--------------|--------------|-----------------|----------|
| intf-subnet  | 192.168.10.3 |              |                 | Accepted |
| intf-subnet  | 192.168.10.2 |              |                 | Accepted |

步驟 13 · 約 5 分鐘

## 13. SWP - Cloud Logging 驗證

您可以查看 Cloud Logging，驗證 SWP 執行的網際網路輸出作業，方法如下：

「監控」→「記錄檔探索工具」

插入查詢：`resource.type="networkservices.googleapis.com/Gateway"` 然後點選「執行查詢」。以下範例會確認目的地端點 `api.frankfurter.app`。



下列 Cloud Logging 範例會驗證以下項目：

`Destination_range`：Agent Engine PSC 介面 IP 位址

`Source_range`：僅限 Proxy 子網路 `Dest_ip`：Secure Web Proxy IP 位址

請務必變更雲端記錄查詢的 `project_id`。

```
logName: ("projects/project_id/logs/compute.googleapis.com%2Ffirewall") AND
jsonPayload.rule_details.reference: ("network:consumer-vpc/firewall:allow-access-to-swp")
```

```
{
  "insertId": "lj9ym95fmu8g6o",
  "jsonPayload": {
    "vpc": {
      "project_id": "XXXXXXXXXXXX",
      "subnetwork_name": "intf-subnet",
      "vpc_name": "consumer-vpc"
    },
    "rule_details": {
      "destination_range": [
        "10.10.10.5/32"
      ],
      "reference": "network:consumer-vpc/firewall:allow-access-to-swp",
      "priority": 1000,
      "source_range": [
        "192.168.10.0/28"
      ],
      "direction": "EGRESS",
      "ip_port_info": [
        {
          "ip_protocol": "ALL"
        }
      ],
      "action": "ALLOW"
    },
    "disposition": "ALLOWED",
    "remote_instance": {
      "region": "us-central1"
    },
    "remote_vpc": {
      "vpc_name": "consumer-vpc",
      "project_id": "XXXXXXXXXXXX",
      "subnetwork_name": "swp-subnet"
    },
    "connection": {
      "src_ip": "192.168.10.2",
      "src_port": 48640,
      "dest_port": 8888,
      "dest_ip": "10.10.10.5",
      "protocol": 6
    }
  },
  "resource": {
    "type": "gce_subnetwork",
    "labels": {
      "subnetwork_id": "7147084067647653041",
      "project_id": "XXXXXXXXXXXX",
      "location": "us-central1",
      "subnetwork_name": "intf-subnet"
    }
  },
  "timestamp": "2025-12-30T12:51:36.628538815Z",
}
```

```
"logName": "projects/dec30-run1-agent/logs/compute.googleapis.com%2Ffirewall",  
"receiveTimestamp": "2025-12-30T12:51:40.846652708Z"  
}
```

---



## 14. 清理

在 JupyterLab 筆記本中，建立新儲存格並執行下列程式碼，觸發刪除 Agent Engine 部署作業。

請務必更新 "project number" 和 "reasoningEngines token"

```
import requests
token = !gcloud auth application-default print-access-token
ENDPOINT = "https://us-central1-aiplatform.googleapis.com"
response = requests.delete(
    f"{ENDPOINT}/v1beta1/projects/218166745590/locations/us-
central1/reasoningEngines/3086854705725308928",
    params={"force": "true"},
    headers={
        "Content-Type": "application/json; charset=utf-8",
        "Authorization": f"Bearer {token[0]}"
    },
)
print(response.text)
```

在 Cloud Shell 中刪除教學課程元件。

注意：Vertex AI Pipeline 至少一小時未使用後，您即可刪除 PSC 網路連結和子網路。

```
gcloud workbench instances delete workbench-tutorial --project=$projectid --location=us-central1-a

gcloud network-security gateway-security-policies rules delete allow-notebook-subnet \
  --gateway-security-policy=my-swp-policy \
  --location=us-central1

gcloud network-security gateway-security-policies rules delete allow-example \
  --gateway-security-policy=my-swp-policy \
  --location=us-central1

gcloud network-security gateway-security-policies delete my-swp-policy \
  --location=us-central1
gcloud network-services gateways delete my-swp-instance\
  --location=us-central1

gcloud dns record-sets delete swp.demo.com --zone=private-dns-codelab --type=A

gcloud dns managed-zones delete private-dns-codelab

gcloud compute network-attachments delete psc-network-attachment --region=us-central1 --quiet

export ROUTER_NAME=$(gcloud compute routers list --regions=us-central1 \
  --filter="name ~ swg-autogen-router" --format="value(name)")

gcloud compute routers nats delete swg-autogen-nat --router=$ROUTER_NAME --region=us-central1 --quiet

gcloud compute routers delete $ROUTER_NAME --region=us-central1 --quiet

gcloud compute networks subnets delete intf-subnet rfc1918-subnet1 --region=us-central1 --quiet

gcloud compute networks subnets delete intf-subnet swp-subnet
  --region=us-central1 --quiet

gcloud compute networks subnets delete intf-subnet intf-subnet --region=us-central1 --quiet

gcloud compute networks subnets delete intf-subnet proxy-subnet
  --region=us-central1 --quiet

gcloud compute networks subnets delete intf-subnet notebook-subnet
  --region=us-central1 --quiet

gcloud compute networks delete consumer-vpc --quiet
```

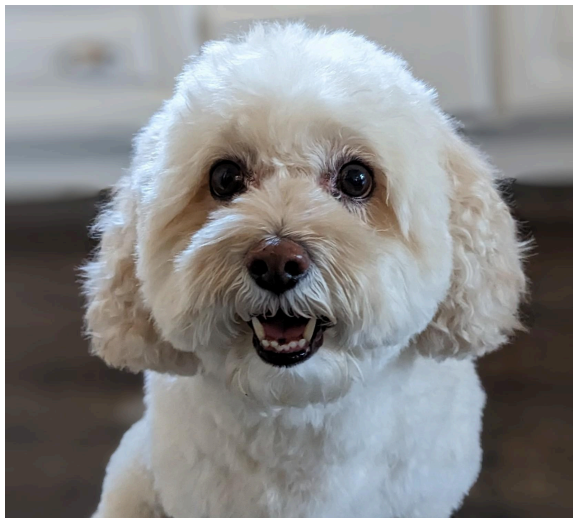
步驟 15 · 約 0 分鐘

## 15. 恭喜

恭喜！您已成功設定並驗證透過 Private Service Connect 介面部署的 Agent Engine，並透過明確的 Proxy 執行網際網路輸出。

您已建立消費者基礎架構，並新增網路附件，讓生產者可以建立多重 NIC VM，以橋接消費者和生產者通訊。您已學會如何建立允許網際網路連線的明確 Proxy 和 DNS 對等互連。

Cosmopup 認為教學課程很棒！



### 後續步驟

### 延伸閱讀和影片

- [Private Service Connect 總覽](#)

## What is Private Service Connect?

Google Cloud Tech

# Private Service Connect

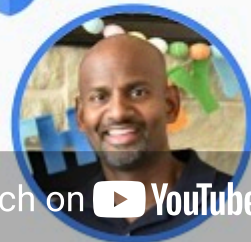


Watch on YouTube

## Vertex AI networking patterns - PSC & Google APIs demo

Advanced Networking Demo

# Connect to Vertex AI using PSC & Googleapis



Watch on YouTube

- [關於 Private Service Connect 介面 | VPC | Google Cloud](#)

## 參考文件

- [Vertex AI 網路存取權總覽 | Google Cloud](#)
- [透過 Private Service Connect 介面存取 Vertex AI 服務 | Google Cloud](#)
- [搭配 Vertex AI Agent Engine 使用 Private Service Connect 介面](#)
- [為 Vertex AI 資源設定 Private Service Connect 介面 | Google Cloud](#)